

std::format() fill character allowances

(Proposed resolution for LWG issues 3576 and 3639)

- [Introduction](#)
- [Changes from P2572R0](#)
- [Design considerations](#)
 - [Character encoding restrictions](#)
 - [Estimated display width restrictions](#)
- [Existing practice](#)
- [Proposal](#)
- [Future considerations and ABI](#)
- [Implementation experience](#)
- [Implementation impact](#)
- [Acknowledgements](#)
- [References](#)
- [Wording](#)

Introduction

Presented is a proposed resolution for the following LWG issues concerning the specification of fill characters in `std::format()`.

- [LWG issue 3576: Clarifying fill character in std::format](#)
- [LWG issue 3639: Handling of fill character width is underspecified in std::format](#)

This proposal follows prior discussion as recorded in the:

- [SG16 meeting summary for 2021-08-25](#).
- [SG16 mailing list archives beginning 2021-11-27 with subject "Agenda for the 2021-12-01 SG16 telecon"](#).
- [SG16 mailing list archives beginning 2021-12-01 with subject "Proposed resolution for LWG3639: Handling of fill character width is underspecified in std::format"](#).
- [SG16 meeting summary for 2021-12-01](#).
- [SG16 mailing list archives beginning 2021-12-02 with subject "Use of U+3000 IDEOGRAPHIC SPACE is common practice"](#).
- [SG16 mailing list archives beginning 2021-12-02 with subject "More Ruminations about fill characters and alignment \(LWG3639\)"](#).
- [LWG minutes for the 2022-11-10 LWG review in Kona](#).
- [LWG minutes for the 2023-02-06 LWG review in Issaquah](#).
- [LWG minutes for the 2023-02-08 LWG review in Issaquah](#).

The current wording in [\[format.string.std\]p1](#) restricts fill characters to "any character other than { or }". Depending on how "character" is interpreted, this may permit characters with a negative display width, characters with no display width, characters with a display width greater than one, characters with a varying display width, characters with an actual display width that differs from their estimated width, combining characters (with or without a non-combining lead character), decomposed characters, characters with right-to-left directionality, control characters, formatting characters, and emoji. The following table presents some examples of such characters.

Glyph	Estimated width	Code point(s)	Character name
>␣<	1	U+0007	BELL
>␣<	1	U+0008	BACKSPACE
><	1	U+007F	DELETE
> ⋄ <	1	U+0009	CHARACTER TABULATION
><	1	U+200B	ZERO WIDTH SPACE
>̂<	1	U+0301	COMBINING ACCUTE ACCENT
>é<	1	U+00E9	LATIN SMALL LETTER E WITH ACUTE
>é<	1	U+0065 U+0301	LATIN SMALL LETTER E COMBINING ACCUTE ACCENT
>ě<	1	U+0065 U+0300 U+0301 U+0302 U+0303 U+0304	LATIN SMALL LETTER E COMBINING GRAVE ACCENT COMBINING ACUTE ACCENT COMBINING CIRCUMFLEX ACCENT COMBINING TILDE COMBINING MACRON
> e <	2	U+FF45	FULLWIDTH LATIN SMALL LETTER E
>エ<	2	U+30A7	KATAKANA LETTER SMALL E
>ㇺ<	1	U+FF6A	HALFWIDTH KATAKANA LETTER SMALL E
> ␣ <	2	U+3000	IDEOGRAPHIC SPACE

>ת<	1	U+05EA	HEBREW LETTER TAV (a right-to-left character)
>🤡<	2	U+1F921	CLOWN FACE
>بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ<	1	U+FDFF	ARABIC LIGATURE BISMILLAH AR-RAHMAN AR-RAHEEM

[*Note*: The glyphs are presented in monospace font and may render inconsistently across browsers and operating systems. The glyphs are displayed between '>' and '<' characters to make it easier to see their presentation width. The estimated width value corresponds to [\[format.string.std\]p11](#); that paragraph associates an estimated width of either one or two with all characters. In each case, the code point sequence constitutes a single extended grapheme cluster. — *end note*]

It is likely that the displayed character differs from the estimated width for at least some cases above; most likely the last case. Unfortunately, there is no specification currently available that governs character display width; actual width may vary based on font selection.

Use of a fill character with a display width other than one potentially prevents a `std::format()` implementation from properly aligning fields. Consider a format specification for a field of width four and a field argument with an estimated field width of one. The implementation is expected to insert fill characters to consume an estimated field width of three, but that is not possible if the fill character has an estimated field width of two. Portable behavior requires that the standard clarify the intended behavior for such characters.

A `std::format()` implementation must store or reference a fill character in some way. Fill character allowances may impose dynamic memory management requirements or increase the complexity of parsing standard format specifiers depending on implementation choices. Implementation choices may also cause fill character restrictions to be reflected in the ABI thus making it difficult to relax restrictions later. Portable behavior requires that the standard specify whether fill characters are restricted to those that are encoded as, for example, a single code unit, a single UCS scalar value, a [stream-safe extended grapheme cluster](#) [\[UAX#15\]](#), or an extended grapheme cluster of unbounded length.

Changes from [P2572R0](#)

- Rebased the proposed wording on N4928.
- Updated the existing practice to reflect the current generation of implementations and added gcc trunk with libstdc++.
- Addressed feedback provided during the 2022-11-10 LWG review in Kona.
- Addressed feedback provided during the 2023-02-06 LWG review in Issaquah.
- Added formal definitions for *field width unit*, *minimum field width*, *estimated field width*, and *padding width*.
- Modified several paragraphs to use the new terms above.
- Replaced the wording for the `0` option for better consistency with the wording for the *align* option.
- Added additional wording to use consistent terminology to refer to the *std-format-spec* grammar elements.
- Removed the proposed wording changes to [22.14.2.2 \[format.string.std\] paragraph 11](#) that replaced the existing uses of "code points" with "UCS scalar values". This change was intended as an unrelated fix, but encountered resistance.
- Added additional drafting notes to better explain the wording changes.

Design considerations

Character encoding restrictions

Fill character allowances pose a performance and overhead tradeoff. Consider the following four options for fill character support.

1. Allow any extended grapheme cluster (EGC).
2. Allow any [stream-safe EGC](#) [\[UAX#15\]](#).
3. Allow any single UCS scalar value.
4. Allow any single UCS scalar value that is encoded using a single code unit.

The first option (any EGC) would require implementations to support EGCs that consist of an unbounded number of code points. This option implies dynamic memory management and would require implementations to identify EGC boundaries in the format string; a requirement that otherwise does not exist at present (implementations are currently required to identify EGC boundaries in formatted field arguments for the purpose of computing the estimated width, but not in the format string itself).

The second option (any stream-safe EGC) would require implementations to support EGCs that consist of up to 32 code points. This option allows an implementation to trade off dynamic memory allocation in favor of larger data structures, but still requires EGC boundary analysis of format strings.

The third option (any single UCS scalar value) avoids dynamic memory requirements and significant increases to sizes of data structures; the fill character could be stored in a single `char32_t` object.

The fourth option (any single code unit) reduces fill character storage requirements to a single code unit (`char` or `wchar_t`), but has the unfortunate side effect of making the permissible set of fill characters dependent on encoding. For example, U+00E9 (LATIN SMALL LETTER E WITH ACUTE) would be rejected in a UTF-8 encoded format string, but would be accepted in a UTF-16 encoded one. Similarly, U+1F921 (CLOWN FACE) would be rejected in a UTF-16 encoded format string, but accepted in a UTF-32 encoded one.

Estimated display width restrictions

The following behaviors represent possible options for formatting fields when the fill character has an estimated width other than one.

1. Use an estimated width of one for the fill character regardless of the value specified in [\[format.string.std\]p11](#).

2. Overfill
- Insert fill characters until the estimated width of the formatted field argument and the fill characters meets or exceeds the field width.
3. Underfill
- Insert fill characters so long as the estimated width of the formatted field argument and the fill characters does not exceed the field width.
4. Pad with a different fill character
- Insert an alternate fill character known to have an estimated width of one when inserting the requested fill character would have caused the field width to be exceeded.
5. Undefined, unspecified, or implementation-defined behavior
- Impose no portable behavior.
6. Error unconditionally
- Throw a `format_error` exception.
7. Error if the alignment to the field width is not possible
- Throw a `format_error` exception if the remainder of the field width after subtracting the estimated width of the formatted field argument is not evenly divisible by the estimated width of the fill character.

The following table illustrates the above options for `std::format(">{:🐼^4}<\n", 'X')`. Font selection will determine to what degree the results shown deviate from the reference alignment.

Behavioral choice	Result
(reference alignment)	>-X--<
Use an estimated width of one	>🐼X🐼🐼<
Overfill	>🐼X🐼<
Underfill	>X🐼<
Pad with a different fill character (space)	> X🐼<
Undefined, unspecified, or implementation-defined behavior	???
Error (unconditionally or due to inability to align)	N/A

Existing practice

The following table illustrates existing behavior for several `std::format()` implementations when the example characters from the [introduction](#) are used as the fill character with a directionally neutral field argument of '#' (the directionality affects the behavior of the U+05EA example). The first row illustrates a reference alignment.

Code point(s)	Format string	Clang 15 with libc++	Gcc 13 trunk with libstdc++	
U+002D HYPHEN-MINUS	">{: - ^4}<"	>-#--<	>-#--<	>-#--<
U+0007 BELL	">{: 🐼 ^4}<"	>🐼#🐼🐼<	>🐼#🐼🐼<	>🐼#🐼🐼<
U+0008 BACKSPACE	">{: ✓ ^4}<"	>✓#✓✓<	>✓#✓✓<	>✓#✓✓<
U+007F DELETE	">{: ^4}<"	>#<	>#<	>#<
U+0009 CHARACTER TABULATION	">{: ^4}<"	>#<	>#<	>#<
U+200B ZERO WIDTH SPACE	">{: ^4}<"	Error ¹	Error ²	>#<
U+0301 COMBINING ACCUTE ACCENT	">{: ´ ^4}<"	Error ¹	Error ²	>#<
U+00E9 LATIN SMALL LETTER E WITH ACUTE	">{: é ^4}<"	Error ¹	Error ²	>é#éé<
U+0065 LATIN SMALL LETTER E U+0301 COMBINING ACCUTE ACCENT	">{: é ^4}<"	Error ¹	Error ²	Error ³
U+0065 LATIN SMALL LETTER E U+0300 COMBINING GRAVE ACCENT U+0301 COMBINING ACUTE ACCENT U+0302 COMBINING CIRCUMFLEX ACCENT U+0303 COMBINING TILDE U+0304 COMBINING MACRON	">{: ẽ ^4}<"	Error ¹	Error ²	Error ³
U+FF45 FULLWIDTH LATIN SMALL LETTER E	">{: e ^4}<"	Error ¹	Error ²	>e#e e<
U+30A7 KATAKANA LETTER SMALL E	">{: ェ ^4}<"	Error ¹	Error ²	>㇐#㇐㇐<
U+FF6A HALFWIDTH KATAKANA LETTER SMALL E	">{: ㇐ ^4}<"	Error ¹	Error ²	>㇐#㇐㇐<
U+3000 IDEOGRAPHIC SPACE	">{: ^4}<"	Error ¹	Error ²	>#<
U+05EA HEBREW LETTER TAV	">{: ת ^4}<"	Error ¹	Error ²	>ת#ת< ⁵ >תחת< ⁵
U+1F921 CLOWN FACE	">{: 🤡 ^4}<"	Error ¹	Error ²	>🤡#🤡🤡<
U+FDFF ARABIC LIGATURE BISMILLAH AR-RAHMAN AR-RAHEEM	">{: بِسْمِ اللّٰهِ الرَّحْمٰنِ الرَّحِیْمِ ^4}<"	Error ¹	Error ²	>والله الرحمن الرحيم

1) Clang with libc++ restricts fill characters to characters that are encoded as a single code unit. Compilation fails with the following error message.

```
error: call to consteval function 'std::basic_format_string<char, char::basic_format_string<char[11]>' is not a constant expression
```

2) Gcc with libstdc++ restricts fill characters to characters that are encoded as a single code point. Compilation fails with the following error message.

```
error: call to non-'constexpr' function 'void std::__format::__failed_to_parse_format_spec()'
```

3) Gcc with fmt restricts fill characters to characters that are encoded as a single code point. Compilation fails with the following error message.

```
error: call to non-'constexpr' function 'void fmt::v9::detail::error_handler::on_error(const char*)'
```

4) MSVC restricts fill characters to characters that are encoded as a single code point. Compilation is successful, but program execution terminates with an exit code of 3221226505 (0xC0000409: STATUS_STACK_BUFFER_OVERRUN). The buffer overflow has been corrected for the next MSVC release and these cases are now rejected with the following error message.

```
error C7595: 'std::_Basic_format_string<char, char>::_Basic_format_string': call to immediate function is not a constant expression
```

5) Use of a fill character with right-to-left directionality potentially causes the formatted field to be rendered right to left depending on the formatted field argument. Two examples are provided, one in which the directionally neutral character '#' is used as the formatted field argument and one in which the left-to-right character 'x' is used. U+200E LEFT-TO-RIGHT MARK characters have been inserted by the paper author to negate the right-to-left effect on surrounding text. In practice, the right-to-left directionality may affect how surrounding text from the format string or other format fields are presented.

All surveyed implementations assume an estimated width of 1 for fill characters regardless of the estimated width values specified in [\[format.string.std\]p11](#).

Proposal

Standardize the behavior exhibited by gcc with fmt and by MSVC:

- Restrict fill characters to a single UCS scalar value.
(This restriction can be lifted in the future if motivation arises for support of EGCs that contain multiple UCS scalar values but may require an ABI break depending on implementation details)
- Always use an estimated width of one for fill characters.
(Ignore [\[format.string.std\]p11](#) when determining how many fill characters to insert)
- Add a note that alignment options have no effect if the estimated width of the formatted field argument exceeds the field width.
- Clarify wording to explicitly describe how fill characters are inserted in order to achieve field alignment.

Future considerations and ABI

Programmers may find use cases where it is necessary for the number of inserted fill characters to depend on the estimated width of the fill character. Some of those use cases may warrant support in the standard. If such motivation arises, there are at least two methods by which support could be added.

1. The [std-format-spec format specification](#) could be extended to allow an additional option to be specified to opt-in to the desired behavior.
2. Specializations of `std::formatter` could be defined to provide custom formatting on a per-type basis as is done for the chrono library.

Motivation may arise in the future to permit the use of an EGC that consists of multiple code points as a fill character. Implementations that store a single `char32_t` or short sequence of code units in their `formatter` class specializations ([\[format.formatter.spec\]](#)) may be unable to accommodate such a change without an ABI break. Implementations are encouraged to instead store a view (an iterator pair, start and end index, or start index and length) into the *std-format-spec* ([\[format.string.std\]p1](#)) string so that code unit sequences of arbitrary length can be referenced. However, since format strings are evaluated at compile-time, there is currently no need for them to be persisted until run-time, so storing a view may impose storage overhead.

It appears that the Microsoft implementation is currently susceptible to such ABI breaks based on the implementation of their [_Basic_format_specs class template](#). Specializations of `_Basic_format_specs` form the base class of their [_Dynamic_format_specs class template](#) for which a specialization is stored in their [Formatter base class template](#) that forms the base class of their [std::formatter specializations](#). Microsoft is already shipping their implementation and is thus already locked into their current ABI.

The author has not researched the ABI break susceptibility of other implementations.

Implementation experience

This proposal standardizes the behavior exhibited by both gcc with fmt and MSVC and therefore reflects existing practice. However the ABI mitigations described in the prior section are not known to have been implemented.

Implementation impact

Some implementations, libstdc++ and libc++ for example, will require changes to allow any single UCS scalar value to be specified as a fill character. This may impose new encoding awareness requirements on format string parsers so that fill characters encoded with more than one code unit are correctly decoded.

Acknowledgements

Thank you to Victor Zverovich, Corentin Jabot, Peter Brett, and Mark de Wever for their insights; their commentary shaped much of this proposal.

References

[N4928] "Working Draft, Standard for Programming Language C++", N4928, 2022.
<https://wg21.link/n4928>

[UAX#15] Ken Whistler,
"Unicode Standard Annex #15 - Unicode Normalization Forms",
Revision 51, Unicode 14.0.0, 2021.
<https://www.unicode.org/reports/tr15/tr15-51.html>

Wording

Drafting note 1: Some intentionally unchanged paragraphs are included in the wording below in order to ease review. These paragraphs are introduced with "No changes to ..." and are [highlighted with a blue background](#).

Drafting note 2: The previous wording was inconsistent with regard to the terminology used when defining and referring to the grammar elements specified in [22.14.2.2 \[format.string.std\] paragraph 1](#). The dominant term used was "option". The proposed wording changes substitute or insert "option" in places where "specifier" or "field" was previously used or where no descriptor was previously present.

Drafting note 3: The wording changes introduce the following new definitions:

- fill character
- field width unit
- minimum field width
- estimated field width
- padding width

Drafting note 4: The following papers contain changes to some of the same paragraphs changed in this paper; merging will be required.

- [P2736 \(Referencing The Unicode Standard\)](#)
- [P2675 \(LWG3780: The Paper\)](#)

These changes are relative to [N4928](#) ^[N4928].

- ☐ Hide inserted text
☐ Hide deleted text

Change in [22.14.2.2 \[format.string.std\] paragraph 1](#):

Each formatter specialization described in [\[format.formatter.spec\]](#) for fundamental and string types interprets *format-spec* as a *std-format-spec*.

[*Note 1:* The format specification can be used to specify such details as [minimum](#) field width, alignment, padding, and decimal precision. Some of the formatting options are only supported for arithmetic types. — *end note*]

The syntax of format specifications is as follows:

std-format-spec:
*fill-and-align*_{opt} *sign*_{opt} *#*_{opt} *θ*_{opt} *width*_{opt} *precision*_{opt} *L*_{opt} *type*_{opt}

fill-and-align:
*fill*_{opt} *align*

fill:
any character other than { or }

align: one of
< > ^

sign: one of
+ - space

width:
positive-integer
{ *arg-id*_{opt} }

precision:
. *nonnegative-integer*
. { *arg-id*_{opt} }

type: one of
a B b C d e E f F g G o p s x X ?

Add a new paragraph after [22.14.2.2 \[format.string.std\].paragraph 1](#):

Field widths are specified in *field width units*; the number of column positions required to display a sequence of characters in a terminal. The *minimum field width* is the number of field width units a replacement field minimally requires of the formatted sequence of characters produced for a format argument. The *estimated field width* is the number of field width units that are required for the formatted sequence of characters produced for a format argument independent of the effects of the *width* option. The *padding width* is the greater of 0 and the difference of the minimum field width and the estimated field width.

[Note 2: The POSIX `wcswidth` function is an example of a function that, given a string, returns the number of column positions required by a terminal to display the string. — end note]

Change in [22.14.2.2 \[format.string.std\].paragraph 2](#):
Drafting note 5: The first sentence of the note was removed because it is redundant; it repeats information present in the grammar for the *fill* option.

The *fill character* is the character denoted by the *fill* option or, if the *fill* option is absent, the space character. For a format specification in a Unicode encoding, the fill character corresponds to a single UCS scalar value.

[Note 2: The *fill* character can be any character other than { or }. The presence of a *fill character* *fill option* is signaled by the character following it, which must be one of the alignment options. If the second character of *std-format-spec* is not a valid alignment option, then it is assumed that both the *fill character* and the *alignment option* are absent. — end note]

Change in [22.14.2.2 \[format.string.std\].paragraph 3](#):

The *align specifier* *option* applies to all argument types. The meaning of the various alignment options is as specified in [Table 66](#).

[Example 1:

```
char c = 120;
string s0 = format("{:6}", 42);           // value of s0 is "    42"
string s1 = format("{:6}", 'x');          // value of s1 is "x     "
string s2 = format("{:*6}", 'x');         // value of s2 is "x*****"
string s3 = format("{:>6}", 'x');         // value of s3 is "*****x"
string s4 = format("{:^6}", 'x');         // value of s4 is "***x***"
string s5 = format("{:6d}", c);           // value of s5 is "   120"
string s6 = format("{:6}", true);         // value of s6 is "true  "
string s7 = format("{:<6.3}", "123456"); // value of s7 is "123***"
string s8 = format("{:02}", 1234);        // value of s8 is "1234"
string s9 = format("{:<}", "12");          // value of s9 is "12"
string sA = format("{:*6}", "12345678"); // value of sA is "12345678"
string sB = format("{:🐼^6}", "x");        // value of sB is "🐼x🐼🐼🐼🐼"
string sC = format("{:*^6}", "🐼🐼🐼🐼"); // value of sC is "🐼🐼🐼🐼🐼🐼"
```

— end example]

[Note 3: Unless a minimum field width is defined, the field width is determined by the size of the content and the alignment option has no effect. The *fill*, *align*, and *0* options have no effect when the minimum field width is not greater than the estimated field width because padding width is 0 in that case. Since fill characters are assumed to have a field width of 1, use of a character with a different field width can produce misaligned output. The 🐼 (U+1F921 CLOWN FACE) character has a field width of 2. The examples above that include that character illustrate the effect of the field width when that character is used as a fill character as opposed to when it is used as a formatting argument. — end note]

Table 66: Meaning of align options [tab:format.align]

Option	Meaning
<	Forces the <i>field formatted argument</i> to be aligned to the start of the <i>available space</i> <i>field</i> by inserting <i>n</i> <i>fill characters</i> after the formatted argument where <i>n</i> is the padding width. This is the default for non-arithmetic non-pointer types, <code>charT</code> , and <code>bool</code> , unless an integer presentation type is specified.
>	Forces the <i>field formatted argument</i> to be aligned to the end of the <i>available space</i> <i>field</i> by inserting <i>n</i> <i>fill characters</i> before the formatted argument where <i>n</i> is the padding width. This is the default for arithmetic types other than <code>charT</code> and <code>bool</code> , pointer types, or when an integer presentation type is specified.
^	Forces the <i>field formatted argument</i> to be centered within the <i>available space</i> <i>field</i> by inserting $\lfloor n/2 \rfloor$ <i>fill characters</i> before and $\lfloor n/2 \rfloor$ <i>fill characters</i> after the formatted argument value, where <i>n</i> is the total number of fill characters to insert the padding width.

No changes to [22.14.2.2 \[format.string.std\].paragraph 4](#):

The *sign* option is only valid ...
[...]

No changes to [22.14.2.2 \[format.string.std\].paragraph 5](#):

The *sign* option applies to ...
[...]

No changes to [22.14.2.2 \[format.string.std\].paragraph 6](#):

The # option causes ...
[...]

Change in [22.14.2.2 \[format.string.std\].paragraph 7](#):

A zero (*0*) character preceding the *width* field pads the field with leading zeros (following any indication of sign or base) to the field width, except when applied to an infinity or NaN. This option is only valid for arithmetic types other than `charT` and `bool` or when an integer presentation type is specified. If the *0* character and an *align* option both appear, the *0* character is ignored.

The *0* option is valid for arithmetic types other than `charT` and `bool` or when an integer presentation type is specified. For formatting arguments that have a value other than an infinity or a NaN, this option pads the formatted argument by inserting the *0* character *n* times following the sign or base prefix indicators (if any), where *n* is *0* if the *align* option is present and is the padding width otherwise.

[Example 3:

```
char c = 120;
double inf = numeric_limits<double>::infinity();
string s1 = format("{:+06d}", c);           // value of s1 is "+00120"
string s2 = format("{:#06x}", 0xa);         // value of s2 is "0x000a"
string s3 = format("{:<06}", -42);          // value of s3 is "-42 " (0 is ignored because of < alignment has no effect)
string s4 = format("{:06}", inf);           // value of s4 is " inf" (0 has no effect).
```

— end example]

Add a new paragraph before [22.14.2.2 \[format.string.std\].paragraph 8](#):

The *width* option specifies the minimum field width. If the *width* option is absent, the minimum field width is *0*.

Change in [22.14.2.2 \[format.string.std\].paragraph 8](#):

If { *arg-id*_{opt} } is used in a *width* or *precision* option, the value of the corresponding formatting argument is used in its place as the value of the option. If the corresponding formatting argument is not of integral type, or its value is negative, an exception of type `format_error` is thrown.

Change in [22.14.2.2 \[format.string.std\].paragraph 9](#):

If positive-integer *n* is used in a *width* option, the value of the decimal integer defining the minimum field width is used as the value of the option. If *width* is not specified, there is no minimum field width, and the field width is determined based on the content of the field.

Remove [22.14.2.2 \[format.string.std\].paragraph 10](#):

Drafting note 6: The content of this paragraph was incorporated into the new paragraph added after paragraph 1.

The *width* of a string is defined as the estimated number of column positions appropriate for displaying it in a terminal.

[Note 5: This is similar to the semantics of the POSIX `wcswidth` function. — end note]

No changes to [22.14.2.2 \[format.string.std\].paragraph 11](#):

For the purposes of width computation, a string is assumed to be in a locale-independent, implementation-defined encoding. Implementations should use a Unicode encoding on platforms capable of displaying Unicode text in a terminal.

[Note 6: This is the case for Windows²⁰⁹-based and many POSIX-based operating systems. — end note]

Change in [22.14.2.2 \[format.string.std\].paragraph 12](#):

For a string in a Unicode encoding, implementations should estimate the width of a string as the sum of estimated widths of the first code points in its extended grapheme clusters. For a sequence of characters in a Unicode encoding, an implementation should use as its field width the sum of the field widths of the first code point of each extended grapheme cluster. The extended grapheme clusters of a string are defined by UAX #29. The estimated width of the following code points is 2: The following code points have a field width of 2:

[...]

The estimated width of field width of all other code points is 1.

Change in [22.14.2.2 \[format.string.std\].paragraph 13](#):

For a string sequence of characters in a non-Unicode encoding, the field width of a string is unspecified.

Change in [22.14.2.2 \[format.string.std\].paragraph 14](#):

The nonnegative integer in *precision* is a decimal integer defining the precision or maximum field size. It can only be used with floating point and string types. For floating point types this field specifies the formatting

precision. For string types, this field provides an upper bound for the estimated width of the prefix of the input string that is copied into the output. For a string in a Unicode encoding, the formatter copies to the output the longest prefix of whole extended grapheme clusters whose estimated width is no greater than the precision. The *precision* option is valid for floating-point and string types. For floating-point types, the value of this option specifies the precision to be used for the floating-point presentation type. For string types, this option specifies the longest prefix of the formatted argument to be included in the replacement field such that the field width of the prefix is no greater than the value of this option.

Add a new paragraph after [22.14.2.2 \[format.string.std\] paragraph 14](#):

Drafting note 7: The wording for this paragraph closely follows paragraph 9.

If *nonnegative-integer* is used in a *precision* option, the value of the decimal integer is used as the value of the option.